Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Пашкевич В. В.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: *приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.*

Задание 1

Порождающий паттерн

Вариант 1

Кофе-автомат с возможностью создания различных кофейных напитков (предусмотреть 5 классов наименований).

Решение

```
from abc import ABC, abstractmethod


# Продукт
class Coffee(ABC):
    @abstractmethod
    def recipe(self) -> str:
        pass

    @abstractmethod
    def price(self) -> float:
        pass

    def prepare(self) -> str:
        return f"Готовится напиток: {self.__class__.__name__}\nРецепт:
{self.recipe()}\nЦена: {self.price()} руб."


# Конкретные продукты
class Espresso(Coffee):
    def recipe(self) -> str:
        return "30 мл воды, 7 г молотого кофе"

    def price(self) -> float:
        return 120.0


class Americano(Coffee):
    def recipe(self) -> str:
        return "Эспрессо + 100 мл горячей воды"

    def price(self) -> float:
        return 140.0


class Cappuccino(Coffee):
    def recipe(self) -> str:
        return "Эспрессо + 120 мл вспененного молока"

    def price(self) -> float:
        return 180.0


class Latte(Coffee):
```

```python
    def recipe(self) -> str:
        return "Эспрессо + 200 мл молока + молочная пена"

    def price(self) -> float:
        return 190.0


class Mocha(Coffee):
    def recipe(self) -> str:
        return "Эспрессо + шоколад + молоко + молочная пена"

    def price(self) -> float:
        return 210.0


# Создатель
class CoffeeFactory(ABC):
    @abstractmethod
    def create_coffee(self) -> Coffee:
        pass


# Конкретные создатели
class EspressoFactory(CoffeeFactory):
    def create_coffee(self) -> Coffee:
        return Espresso()


class AmericanoFactory(CoffeeFactory):
    def create_coffee(self) -> Coffee:
        return Americano()


class CappuccinoFactory(CoffeeFactory):
    def create_coffee(self) -> Coffee:
        return Cappuccino()


class LatteFactory(CoffeeFactory):
    def create_coffee(self) -> Coffee:
        return Latte()


class MochaFactory(CoffeeFactory):
    def create_coffee(self) -> Coffee:
        return Mocha()


# Клиент
class CoffeeMachine:
    def __init__(self):
        self.factories = {
            "espresso": EspressoFactory(),
            "americano": AmericanoFactory(),
            "cappuccino": CappuccinoFactory(),
            "latte": LatteFactory(),
            "mocha": MochaFactory(),
        }

    def make_coffee(self, coffee_name: str) -> Coffee:
        coffee_name = coffee_name.lower()

        factory = self.factories.get(coffee_name)
        if not factory:
```

```
            raise ValueError(f"Напиток '{coffee_name}' не поддерживается
автоматом.")

        coffee = factory.create_coffee()
        print(coffee.prepare())
        return coffee


# Демонстрация работы
if __name__ == "__main__":
    machine = CoffeeMachine()

    orders = ["espresso", "latte", "mocha", "americano", "cappuccino"]

    for order in orders:
        print("=" * 50)
        machine.make_coffee(order)
```

```
==================================================
Готовится напиток: Espresso
Рецепт: 30 мл воды, 7 г молотого кофе
Цена: 120.0 руб.
==================================================
Готовится напиток: Latte
Рецепт: Эспрессо + 200 мл молока + молочная пена
Цена: 190.0 руб.
==================================================
Готовится напиток: Mocha
Рецепт: Эспрессо + шоколад + молоко + молочная пена
Цена: 210.0 руб.
==================================================
Готовится напиток: Americano
Рецепт: Эспрессо + 100 мл горячей воды
Цена: 140.0 руб.
==================================================
Готовится напиток: Cappuccino
Рецепт: Эспрессо + 120 мл вспененного молока
Цена: 180.0 руб.
```

Задание 2

Структурный паттерн

Вариант 1

Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

Решение

```
from abc import ABC, abstractmethod


class DigitalClockInterface(ABC):
    @abstractmethod
    def set_time(self, hours: int, minutes: int, seconds: int) -> None:
```

```
        """Установить время."""
        raise NotImplementedError

    @abstractmethod
    def get_time(self) -> str:
        """Получить время в цифровом формате."""
        raise NotImplementedError


class AnalogClock:

    def __init__(self) -> None:
        self.hour_angle = 0.0
        self.minute_angle = 0.0
        self.second_angle = 0.0

    def set_angles(
        self,
        hour_angle: float,
        minute_angle: float,
        second_angle: float,
    ) -> None:
        """Установить углы стрелок."""
        self.hour_angle = hour_angle % 360
        self.minute_angle = minute_angle % 360
        self.second_angle = second_angle % 360

    def get_angles(self) -> tuple[float, float, float]:
        """Получить углы стрелок."""
        return self.hour_angle, self.minute_angle, self.second_angle


class ClockAdapter(DigitalClockInterface):
    """Адаптер, позволяющий работать с аналоговыми часами как с цифровыми."""

    def __init__(self, source_clock: AnalogClock) -> None:
        self.source_clock = source_clock

    def set_time(self, hours: int, minutes: int, seconds: int) -> None:
        """Установить время в формате часы:минуты:секунды."""
        if not 0 <= hours <= 23:
            raise ValueError("Часы должны быть в диапазоне 0..23")
        if not 0 <= minutes <= 59:
            raise ValueError("Минуты должны быть в диапазоне 0..59")
        if not 0 <= seconds <= 59:
            raise ValueError("Секунды должны быть в диапазоне 0..59")

        normalized_hours = hours % 12

        second_angle = seconds * 6
        minute_angle = minutes * 6 + seconds * 0.1
        hour_angle = normalized_hours * 30 + minutes * 0.5 + seconds * (0.5 /
60)

        self.source_clock.set_angles(hour_angle, minute_angle, second_angle)

    def get_time(self) -> str:
        """Получить время в цифровом виде."""
        hour_angle, minute_angle, second_angle =
self.source_clock.get_angles()

        seconds = round(second_angle / 6) % 60
        minutes = int(minute_angle / 6) % 60
        hours = int(hour_angle / 30) % 12
```

```
        return f"{hours:02d}:{minutes:02d}:{seconds:02d}"

    def get_full_info(self) -> str:
        """Получить время и углы стрелок."""
        hour_angle, minute_angle, second_angle =
self.source_clock.get_angles()
        return (
            f"Текущее цифровое время: {self.get_time()}\n"
            f"Углы стрелок:\n"
            f"  Часовая стрелка: {hour_angle:.2f}°\n"
            f"  Минутная стрелка: {minute_angle:.2f}°\n"
            f"  Секундная стрелка: {second_angle:.2f}°"
        )


if __name__ == "__main__":
    clock_model = AnalogClock()
    clock_adapter = ClockAdapter(clock_model)

    clock_adapter.set_time(10, 25, 30)
    print(clock_adapter.get_full_info())

    print("\nИзменим время:")
    clock_adapter.set_time(3, 45, 0)
    print(clock_adapter.get_full_info())
```

```
Текущее цифровое время: 10:25:30
Углы стрелок:
  Часовая стрелка: 312.75°
  Минутная стрелка: 153.00°
  Секундная стрелка: 180.00°

Изменим время:
Текущее цифровое время: 03:45:00
Углы стрелок:
  Часовая стрелка: 112.50°
  Минутная стрелка: 270.00°
  Секундная стрелка: 0.00°
```

Задание 3

Поведенческий паттерн

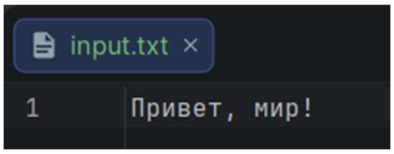
Вариант 9

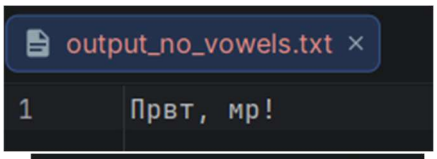
Шифрование текстового файла. Реализовать класс-шифровщик текстового файла с поддержкой различных алгоритмов шифрования. Возможные варианты шифрования: удаление всех гласных букв из текста, изменение букв текста на буквы, получаемые фиксированным сдвигом из алфавита (например, шифром буквы а будет являться буква д для сдвига 4 и т.д.), применение операции исключающее или с заданным ключом.

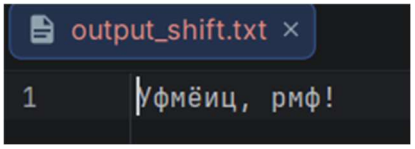
Решение

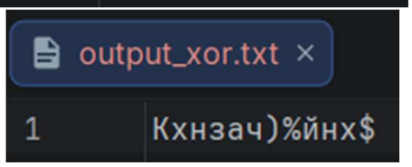
```python
from abc import ABC, abstractmethod


class EncryptionStrategy(ABC):
    """Абстрактная стратегия шифрования."""

    @abstractmethod
    def encrypt(self, text: str) -> str:
        """Зашифровать текст."""
        raise NotImplementedError


class RemoveVowelsStrategy(EncryptionStrategy):
    """Стратегия удаления всех гласных букв из текста."""

    def encrypt(self, text: str) -> str:
        vowels = "аеёиоуыэюяАЕЁИОУЫЭЮЯaeiouAEIOU"
        return "".join(char for char in text if char not in vowels)


class ShiftCipherStrategy(EncryptionStrategy):

    def __init__(self, shift: int) -> None:
        self.shift = shift
        self.ru_lower = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"
        self.ru_upper = self.ru_lower.upper()
        self.en_lower = "abcdefghijklmnopqrstuvwxyz"
        self.en_upper = self.en_lower.upper()

    def _shift_char(self, char: str, alphabet: str) -> str:
        index = alphabet.index(char)
        new_index = (index + self.shift) % len(alphabet)
        return alphabet[new_index]

    def encrypt(self, text: str) -> str:
        result = []

        for char in text:
            if char in self.ru_lower:
                result.append(self._shift_char(char, self.ru_lower))
            elif char in self.ru_upper:
                result.append(self._shift_char(char, self.ru_upper))
            elif char in self.en_lower:
                result.append(self._shift_char(char, self.en_lower))
            elif char in self.en_upper:
                result.append(self._shift_char(char, self.en_upper))
            else:
                result.append(char)

        return "".join(result)


class XorCipherStrategy(EncryptionStrategy):

    def __init__(self, key: int) -> None:
        self.key = key

    def encrypt(self, text: str) -> str:
        encrypted_chars = [chr(ord(char) ^ self.key) for char in text]
        return "".join(encrypted_chars)
```

```python
class TextFileEncryptor:
    """Класс-шифровщик текстовых файлов."""

    def __init__(self, strategy: EncryptionStrategy) -> None:
        self.strategy = strategy

    def set_strategy(self, strategy: EncryptionStrategy) -> None:
        """Сменить алгоритм шифрования."""
        self.strategy = strategy

    def encrypt_file(self, input_file: str, output_file: str) -> None:
        """Считать текст из файла, зашифровать и записать в другой файл."""
        with open(input_file, "r", encoding="utf-8") as file:
            text = file.read()

        encrypted_text = self.strategy.encrypt(text)

        with open(output_file, "w", encoding="utf-8") as file:
            file.write(encrypted_text)


if __name__ == "__main__":
    input_filename = "input.txt"

    # 1. Удаление гласных
    encryptor = TextFileEncryptor(RemoveVowelsStrategy())
    encryptor.encrypt_file(input_filename, "output_no_vowels.txt")

    # 2. Сдвиг на 4 символа
    encryptor.set_strategy(ShiftCipherStrategy(4))
    encryptor.encrypt_file(input_filename, "output_shift.txt")

    # 3. XOR с ключом 5
    encryptor.set_strategy(XorCipherStrategy(5))
    encryptor.encrypt_file(input_filename, "output_xor.txt")

    print("Шифрование завершено.")
```









Вывод: *приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.*